CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA
FACULDADE DE TECNOLOGIA DA BAIXADA SANTISTA "RUBENS LARA"
CURSO SUPERIOR DE TECNOLOGIA EM CIÊNCIA DE DADOS
PROGRAMA DE INICIAÇÃO CIENTÍFICA

PETERSON CARARA JUNIOR

FORMALIZAÇÃO DE MÁQUINAS DE ESTADOS FINITOS COMPONÍVEIS EM HASKELL: SEGURANÇA DE TIPOS E COMPLETUDE FUNCIONAL VIA MONADS E TIPOS ALGÉBRICOS

Santos - SP
2025

PETERSON CARARA JUNIOR

FORMALIZAÇÃO DE MÁQUINAS DE ESTADOS FINITOS COMPONÍVEIS EM HASKELL: SEGURANÇA DE TIPOS E COMPLETUDE FUNCIONAL VIA MONADS E TIPOS ALGÉBRICOS

Projeto de pesquisa apresentado à Faculdade de Tecnologia da Baixada Santista “Rubens Lara”, do Curso Superior de Tecnologia em Ciência de Dados, no Programa de Iniciação Científica.

Orientador: Prof. Felipe Canarozzo Lourenço

Santos - SP
2025

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ADT	Algebraic Data Type (Tipo de Dados Algébrico)
CREM	Crème de la Crem: Composable Representable Executable Machines
DDD	Domain-Driven Design
DFSM	Deterministic Finite State Machine
FATEC BS	Faculdade de Tecnologia da Baixada Santista Rubens Lara
FSM	Finite State Machine (Máquina de Estados Finitos)
I/O	Input/Output
PFP	Programação Funcional Pura

Sumário

LISTA DE ABREVIATURAS E SIGLAS 1

1 INTRODUÇÃO 4
1.1 PROBLEMA DE PESQUISA 5
1.2 JUSTIFICATIVA DO TEMA 5
1.2.1 Hipóteses 5
1.2.2 Objetivo Geral 6
1.2.3 Objetivos Específicos 6

2 REFERENCIAL TEÓRICO INICIAL 7
2.1 Máquinas de Estados Finitos e a Questão da Mutabilidade 7
2.2 Haskell e a Garantia de Corretude em Tempo de Compilação 7
2.2.1 Tipos de Dados Algébricos (ADTs) 8
2.2.2 Pattern Matching e Completude Funcional 8
2.3 Teoria das Categorias: Monads, Estado Puro e a Categoria Kleisli 8
2.3.1 Monads: Origem Categorial, Leis e a Abstração do Efeito de Estado . . 8
2.3.2 A Categoria Kleisli para a Composição de Transições 9
2.4 Composabilidade Arquitetural e o Framework CREM 9

3 PROCEDIMENTOS METODOLÓGICOS 10

4 ESTADO DA ARTE 11

5 PROGRESSO PARCIAL 11
5.1 Correspondência entre Tipos de Dados Algébricos e os Conjuntos Q e Sigma . 12
5.2 A Função de Transição delta como Morfismo em Hask 12
5.3 Sequenciação Inicial e Validação Empírica 13
5.4 Da Composição Ingênua à Necessidade de uma Abstração Monádica 14
5.5 Reformulação Teórica da FSM sobre a State Monad 14
5.6 Sequenciação via Notação do e a Função mapM 15
5.7 Fundamentação Categorial: o Monad como Monoide na Categoria dos Endofuntores 16
5.8 Estudo de Caso: Topologia Promovida ao Nível de Tipos 16
5.9 Verificação Formal de Segurança de Tipos em Tempo de Compilação 17
5.10 Execução e Validação Cruzada 18
5.11 Limitações Identificadas e Encaminhamentos 18
5.12 Síntese Geral e Validação Preliminar das Hipóteses 19

6 CRONOGRAMA 20

7 REFERÊNCIAS 21

1 INTRODUÇÃO

As Máquinas de Estados Finitos (FSMs) constituem uma das abstrações mais duradouras e essenciais da Ciência da Computação. Elas fornecem o formalismo matemático rigoroso para a modelagem de sistemas cujo comportamento é estritamente determinado por um conjunto discreto de estados e regras de transição. O modelo é crucial para a especificação do comportamento de sistemas reativos, abrangendo desde protocolos de rede até a lógica de domínio em arquiteturas de software complexas. Conforme Perone (2023), é possível construir arquiteturas de software a partir da composição de máquinas de estados.

No entanto, a tradução desse modelo abstrato para linguagens de programação de propósito geral do paradigma imperativo, como C++, é frequentemente marcada pela dependência de variáveis mutáveis, tipicamente campos de classe ou variáveis globais, para armazenar o estado atual. Essa prática compromete a pureza do modelo matemático. Segundo Malakhovski e Shalyto (2009), a implementação em linguagens imperativas não permite a aplicação direta dos métodos tradicionais de FSMs em um ambiente funcional. A dependência da mutabilidade e de tipos fracos para o estado expõe o sistema a erros lógicos e transições inválidas, delegando a responsabilidade de verificar a corretude à fase de testes em tempo de execução.

O presente projeto de pesquisa propõe resolver essa discrepância fundamental através da **formalização e implementação de FSMs em Haskell**. Por ser uma linguagem puramente funcional (Hutton, 2016), Haskell exige que a função de transição seja implementada como uma função pura. O diferencial reside no seu sistema de tipos avançado: a utilização de **Tipos de Dados Algébricos (ADTs)** é essencial para a modelagem de dados (Milewski, 2020), permitindo que os conjuntos finitos de estados e eventos sejam mapeados diretamente para tipos de dados. Essa correspondência de tipo não é apenas uma convenção, mas uma imposição que, por meio da verificação de exaustividade do *pattern matching*, viabiliza a **verificação da correção** das funções de transição no estágio de compilação (Malakhovski & Shalyto, 2009).

Além da segurança de tipos básica, a abordagem em Haskell permite a exploração de conceitos avançados para o tratamento do estado. A capacidade de sequenciar transições e lidar com o estado de forma pura é resolvida através de **Monads**. Essa abstração, que expressa de forma concisa programas que manipulam estado (Wadler, 1992), permite o tratamento **puro do estado** através de estruturas como a `State Monad` (Milewski, 2020). Esse arcabouço monádico, estruturado pela **Categoria Kleisli** (Maia, 2022), fornece o formalismo necessário para a **composição** segura de transições.

A capacidade de **composabilidade** é o ponto culminante desta abordagem. O *framework* **Crem** (Crème de la Crem: Composable Representable Executable Machines), desenvolvido por Marco Perone (2023), ilustra como os *advanced type-level features* de Haskell podem ser usados para especificar transições de estado permitidas e proibidas. Essa especificação transforma as regras de domínio em propriedades intrínsecas ao tipo de dados, permitindo a **composição de máquinas de estados** independentes (Perone, 2023), resultando em um sistema robusto e

modular, onde a lógica de domínio é validada pelo compilador antes da execução.

1.1 PROBLEMA DE PESQUISA

Considerando o descompasso entre a formalidade matemática das FSMs e a fragilidade de tipos nas implementações tradicionais (Malakhovski & Shalyto, 2009), o problema de pesquisa é formulado como:

> De que forma a linguagem de programação funcional pura **Haskell**, explorando seu sistema de tipos avançados (**Tipos de Dados Algébricos**) e suas abstrações de ordem superior (**Monads** e **Categoria Kleisli**), pode ser utilizada para obter uma formalização de Máquinas de Estados Finitos que ofereça garantias verificáveis de correção (*type safety*) e completude (*completeness*) na especificação das funções de transição, e que suporte a composição de **FSMs complexas** de maneira modular e representável?

1.2 JUSTIFICATIVA DO TEMA

A relevância do projeto reside em transferir a verificação da corretude da FSM da fase de testes para a fase de compilação. Segundo Malakhovski e Shalyto (2009), uma das vantagens da implementação funcional é a verificação da correção do código das funções de transição já no estágio de compilação, o que é possível pelo uso de **Tipos de Dados Algébricos (ADTs)** para representar eventos e estados.

Além disso, a estrutura de Haskell é baseada na **Teoria das Categorias** (Milewski, 2020). Essa base formal permite lidar com o estado de forma pura através de **Monads**, que foram originalmente desenvolvidas por teóricos de categorias para expressar concisamente certos aspectos da álgebra universal (Wadler, 1992). Formalmente, uma monad é definida como um **Monóide na Categoria dos Endofuntores** (Jahn, 2020) e permite a mudança de estado de forma pura (Milewski, 2020). Essas estruturas possibilitam a composição segura de transições via a **Categoria Kleisli** (Maia, 2022; Jahn, 2020). Essa composabilidade é crucial, pois permite a construção de arquiteturas de software a partir da composição de máquinas de estados independentes (Perone, 2023), o que contribui para a modularidade e a coerência do domínio do *software*.

1.2.1 Hipóteses

O sistema de tipos de Haskell oferece uma **Segurança de Tipos** superior, onde o uso dos Tipos de Dados Algébricos (ADTs) e o *pattern matching* conseguem impor, em tempo de compilação, a proibição de estados e transições logicamente impossíveis. Essa garantia se estende à **Completude Funcional**, onde o *pattern matching* exaustivo assegura que a função de transição cubra todas as combinações válidas de (Estado $\times$ Evento), o que é uma superioridade na

verificação da lógica da FSM em relação às linguagens imperativas (Malakhovski & Shalyto, 2009). Finalmente, postula-se a **Hipótese de Composabilidade e Modulação**: a aplicação de abstrações categoriais como **Monads** e recursos avançados de *type-level* de Haskell (como os utilizados na biblioteca Crem) permite a criação de **Máquinas de Estados Finitos Componíveis** de forma puramente funcional, resultando em uma arquitetura modular cujas regras de domínio são validadas pelo compilador (Perone, 2023).

1.2.2 Objetivo Geral

O objetivo geral desta Iniciação Científica é formalizar e implementar **Máquinas de Estados Finitos Determinísticas (DFSMs)** em **Haskell**, explorando o sistema de tipagem avançado e as estruturas funcionais da linguagem para garantir a segurança e a completude na definição das funções de transição, com ênfase na demonstração de um modelo que suporte a composição de sistemas.

1.2.3 Objetivos Específicos

Para alcançar o objetivo geral de formalizar e implementar FSMs de forma segura e componível em Haskell, o projeto se desdobra nos seguintes objetivos específicos.

O primeiro objetivo visa o alicerce do projeto: **definir modelos de dados em Haskell** que traduzam com precisão os elementos formais de uma FSM. Esta formalização será realizada através da criação de **Tipos de Dados Algébricos (ADTs)**, nos quais os conjuntos finitos de Estados (Q) e Eventos (Σ) serão mapeados diretamente para Tipos Soma. Essa técnica visa incorporar o rigor do modelo matemático ao código, seguindo o princípio de que ADTs são o conceito mais importante na modelagem funcional (Milewski, 2020).

O segundo objetivo trata da validação da lógica interna da máquina: **implementar o mecanismo de processamento de sequências de entrada e a função de transição** (δ). Serão exploradas as funções de alta ordem e, sobretudo, o ***pattern matching*** exaustivo para definir a função $\delta : \text{Estado} \times \text{Evento} \to \text{Estado}$. Essa abordagem é fundamental para atingir a **Hipótese de Completude Funcional**, pois o compilador de Haskell garantirá a exaustividade, assegurando que todo par válido $(\text{Estado}, \text{Evento})$ seja tratado, transferindo a verificação da correção do código das funções de transição para o estágio de compilação (Malakhovski & Shalyto, 2009).

Em seguida, o terceiro objetivo foca na gestão do estado e na composabilidade: **explorar a aplicação do *State Monad* e da Categoria Kleisli**. Será demonstrado como a `State Monad` permite gerenciar a sequência de mudanças de estado de forma pura, encapsulando o estado e garantindo a mudança de estado de forma pura (Milewski, 2020). Esta etapa é crítica para lidar com o histórico de transições. Além disso, a **Categoria Kleisli** será utilizada como base teórica para **compor transições monádicas**, fornecendo o formalismo para encadear logicamente as transições de estado, como sugerido por Milewski (2020) e Maia (2022).

Finalmente, o quarto objetivo é a validação prática da abordagem: **demonstrar a composabilidade de máquinas de estados por meio de um estudo de caso**. Este estudo de caso ilustrará como a tipagem de Haskell pode ser usada para garantir transições válidas e o encapsulamento do domínio, permitindo que a lógica do *software* seja estruturada como uma composição de máquinas de estados modulares (Perone, 2023). O resultado esperado é um sistema onde as regras de transição — inclusive a especificação de transições de estado permitidas e proibidas (Perone, 2023) — são validadas pelo compilador, confirmando o potencial das estruturas funcionais para o desenvolvimento de *software* robusto.

2 REFERENCIAL TEÓRICO INICIAL

O presente referencial teórico estabelece a base de conhecimento necessária para a formalização das **Máquinas de Estados Finitos (FSMs)** no ambiente **funcional puro**. A abordagem é construída sobre a fusão de quatro domínios conceituais: a **Teoria de Autômatos**, a **Programação Funcional Pura (PFP)**, a **Teoria das Categorias** e as **Arquiteturas de *Software* Componíveis**.

2.1 Máquinas de Estados Finitos e a Questão da Mutabilidade

As FSMs são o modelo canónico para descrever sistemas sequenciais e reativos, cuja dinâmica é regida por uma **quíntupla formal** $(Q, \Sigma, \delta, q_0, F)$, onde Q é o conjunto finito de estados, Σ é o alfabeto de entrada (eventos), δ é a rigorosa função de transição, q_0 é o estado inicial, e F é o conjunto de estados finais. A função $\delta : Q \times \Sigma \to Q$ é o coração do sistema, definindo como o autômato se move entre os estados.

A implementação tradicional desse modelo em linguagens de programação imperativas, como C++ ou Java, cria uma divergência perigosa entre o modelo e o código. A representação do estado recorre, invariavelmente, a variáveis mutáveis — tipicamente um campo de classe (se a programação orientada a objetos for usada) ou uma variável global (Malakhovski & Shalyto, 2009). Além disso, os identificadores de estado são frequentemente representados por abstrações de baixo nível, como *strings* ou inteiros, que carecem de garantias de tipo. Essa dependência de estado mutável e a fragilidade de tipos resultam numa tradução que não consegue aplicar diretamente os métodos utilizados na implementação de máquinas de estados finitos em linguagens de programação imperativas no rigoroso domínio funcional (Malakhovski & Shalyto, 2009), expondo a FSM a estados logicamente impossíveis ou transições inválidas que só são detectadas em tempo de execução, através de pesados testes unitários.

2.2 Haskell e a Garantia de Corretude em Tempo de Compilação

O projeto propõe a **Programação Funcional Pura (PFP)** como a solução para este descompasso. **Haskell**, uma linguagem puramente funcional, baseia-se na imutabilidade e na ausência de efeitos colaterais. Sua filosofia exige que a função de transição (δ) seja implementada como

uma **função pura**, uma transformação que recebe o estado e o evento e retorna, deterministicamente, o novo estado, preservando a imutabilidade do sistema e produzindo *software* **claro, conciso e correto** (Hutton, 2016).

2.2.1 Tipos de Dados Algébricos (ADTs)

A segurança formal em Haskell é fornecida por seu sistema de tipos avançado, ancorado nos **Tipos de Dados Algébricos (ADTs)**. Os ADTs permitem que os conjuntos finitos Q e Σ da FSM sejam mapeados diretamente para **Tipos Soma**, que são considerados o conceito mais importante para a modelagem em Haskell (Milewski, 2020). Por exemplo, se uma FSM possui estados *Ligado* e *Desligado*, o Tipo Soma **data Estado = Ligado | Desligado** traduz essa finitude para o sistema de tipos. Essa codificação evita o uso de **strings e inteiros** como identificadores de estado (Malakhovski & Shalyto, 2009), garantindo que o compilador imponha a integridade estrutural da máquina.

2.2.2 Pattern Matching e Completude Funcional

A função de transição é implementada por meio de ***pattern matching***, uma funcionalidade poderosa para examinar a estrutura dos dados (Hutton, 2016). O aspecto crucial é que o compilador de Haskell executa uma **verificação de exaustividade** neste *pattern matching*. Essa checagem garante que toda combinação possível de $(\text{Estado} \times \text{Evento})$ tenha sido tratada explicitamente na definição da função δ. Caso uma combinação válida seja omitida, o compilador alerta o programador com um erro, o que se traduz na **verificação da correção do código das funções de transição no estágio de compilação** (Malakhovski & Shalyto, 2009), confirmando a **completude (*completeness*)** da FSM antes da execução.

2.3 Teoria das Categorias: Monads, Estado Puro e a Categoria Kleisli

Em um ambiente funcional puro, onde as transições de estado devem ser implementadas como funções puras, o desafio inerente é a gestão da **sequência de transições** ao longo do tempo. O projeto aborda este desafio recorrendo à **Teoria das Categorias**, que é a disciplina matemática que fornece os fundamentos lógicos para a linguagem Haskell. O princípio orientador é que a composição é a essência da programação (Milewski, 2020), e a Teoria das Categorias define o arcabouço para a composição segura e coerente.

2.3.1 Monads: Origem Categorial, Leis e a Abstração do Efeito de Estado

As **Monads** são o dispositivo arquitetónico que permite a **Programação Funcional Pura** gerir efeitos colaterais, como o estado, exceções ou I/O, sem violar a imutabilidade. Conforme Philip Wadler (1992) explica, Monads proporcionam as indulgências de impureza com as bênçãos de pureza ao encapsular o "efeito".

A sua origem é estritamente matemática: Monads foram inventadas na década de 1960 por teóricos de categorias para expressar concisamente certos aspectos da álgebra universal (Wadler, 1992). A definição categorial formal e mais abstrata estabelece uma Monad como um **Monóide na Categoria dos Endofuntores** (Jahn, 2020). Esta definição, embora abstrata, é fundamental, pois garante que a estrutura Monádica satisfaça as **Leis das Monads** (identidade e associatividade), assegurando que o encadeamento de efeitos seja previsível e deterministicamente seguro.

Para a FSM, a **`State Monad`** é a Monad específica empregada. Ela resolve o dilema do estado ao modelar a transição como uma função de transformação de estado, na qual a função de transição pura δ é encapsulada. Isto permite que a **mudança de estado de forma pura** seja realizada (Milewski, 2020), pois a função não *modifica* o estado, mas sim *calcula* o novo estado a partir do antigo, retornando o par (Valor, Novo Estado) e mantendo a imutabilidade fora da abstração monádica.

2.3.2 A Categoria Kleisli para a Composição de Transições

Embora Monads resolvam o problema do estado encapsulado, a composição dessas funções monádicas (as transições de estado) não pode ser realizada pela composição de funções padrão. É aqui que entra a **Categoria Kleisli**.

A Kleisli é uma construção categorial específica que é projetada para compor **morfismos monádicos**, ou seja, funções "sujas"que carregam o efeito do *Monad*. Em vez de funções $A \to B$, a Kleisli compõe morfismos do tipo $A \to MB$ (Maia, 2022), onde M é o Functor Monádico (a `State Monad` no nosso caso) que carrega o estado. Essa categoria fornece as regras e, mais importante, o operador de composição monádica (>=>) que automatiza a logística da sequenciação.

O uso da Categoria Kleisli é crucial para o projeto, pois ele fornece o formalismo para encadear múltiplas transições de estado da FSM de forma segura e elegante. A Monad, sob a estrutura da Kleisli, garante que o estado do autômato seja automaticamente passado, transformado e extraído em cada etapa da sequência de eventos. Isso não só simplifica a sintaxe, como garante, a partir dos fundamentos da **Teoria das Categorias**, que a composição de transições monádicas seja sempre **coerente** e **associativa**, um pré-requisito fundamental para a **composabilidade** de FSMs complexas em arquiteturas modulares.

2.4 Composabilidade Arquitetural e o Framework CREM

A validação final da abordagem reside na sua capacidade de suportar arquiteturas de *software* complexas. O projeto alinha-se aos princípios do *Domain-Driven Design* (**DDD**), utilizando a FSM como um agregado de domínio verificável.

O *framework* **Crème de la Crem (CREM)** é um exemplo prático que demonstra o potencial máximo da PFP nesse contexto. O CREM utiliza os **recursos avançados de *type-level* de**

Haskell para permitir que o *software* seja estruturado como uma composição de máquinas de estados independentes (Perone, 2023). Essa modularidade permite que subsistemas de domínio sejam desenvolvidos de forma isolada e testada, e combinados de forma segura.

A maior contribuição do CREM é a capacidade de **codificar regras de negócio como tipos**. Ele permite ao usuário especificar transições de estado permitidas e proibidas diretamente na assinatura de tipos (Perone, 2023). Isso significa que as restrições mais complexas da lógica de domínio da FSM são transformadas em restrições de tipo, e qualquer violação de uma regra de negócio resultará em um erro de compilação, e não em um erro de *runtime*. Isso garante que a implementação de um sistema se mantenha sincronizada com a sua representação gráfica (Perone, 2023), estabelecendo um nível de **representabilidade** e integridade arquitetural superior, onde a lógica de negócio é verificada pelo compilador.

3 PROCEDIMENTOS METODOLÓGICOS

A metodologia deste projeto será de natureza mista, combinando o rigor da pesquisa formal com a validação prática, pautando-se numa **abordagem qualitativa** para analisar a fundo o fenômeno da formalização de Máquinas de Estados Finitos (FSMs) no contexto da programação funcional pura. O trabalho se desenvolverá em duas vertentes principais: a **Pesquisa Bibliográfica** e a **Pesquisa Aplicada (ou Tecnológica)**. A Pesquisa Bibliográfica fornecerá a base formal, coletando e analisando o referencial teórico sobre FSMs, Programação Funcional Pura (Haskell), Tipos de Dados Algébricos (ADTs), Teoria das Categorias (Monads, Categoria Kleisli) e os princípios do *framework* CREM. Esta base teórica é crucial para a construção do modelo computacional.

A **Pesquisa Aplicada** será a fase de experimentação, focada no desenvolvimento de um artefato de *software* (código em Haskell) para comprovar as hipóteses de segurança e composabilidade. Os procedimentos serão organizados em quatro fases interligadas que visam, em última instância, cumprir todos os objetivos específicos do projeto.

A **Fase 1: Revisão Teórica e Formalização Inicial** visa estabelecer o alicerce do código. Inicialmente, será concluída a revisão aprofundada dos fundamentos da FSM e de como a imutabilidade da PFP se traduz numa alternativa superior ao modelo imperativo. Em seguida, o primeiro objetivo específico será atendido pela **modelagem de ADTs**, onde os conjuntos de Estados (Q) e Eventos (Σ) de uma FSM canónica serão declarados como Tipos Soma em Haskell. O segundo objetivo será alcançado com a **implementação da função de transição pura** δ utilizando ***pattern matching*** exaustivo, o que é um procedimento metodológico chave para validar a **Hipótese de Completude Funcional**, demonstrando que o compilador exige o tratamento de todas as combinações válidas $(\text{Estado} \times \text{Evento})$.

A **Fase 2: Gestão do Estado e Sequenciação Monádica** endereçará o desafio da pureza na manipulação de estado, cumprindo o terceiro objetivo específico. O estudo da `State Monad` será traduzido na implementação, garantindo que a sequência de transições seja realizada atra-

vés de uma função que calcula o novo estado a partir do antigo, mantendo a **mudança de estado de forma pura** (Milewski, 2020). Esta fase incluirá a sequenciação de eventos utilizando a notação *do-notation* de Haskell, demonstrando como a **Categoria Kleisli** fornece o formalismo necessário para a composição segura dessas transições monádicas.

A **Fase 3: Demonstração da Composabilidade e Segurança** é a fase de validação prática da arquitetura. Será selecionado um **Estudo de Caso de Domínio** que exija a composição de múltiplas FSMs independentes. O procedimento metodológico aqui será aplicar os conceitos da Categoria Kleisli para compor as máquinas, provando a modularidade arquitetural. O ponto de maior rigor será o **Teste de Invalidez de Tipagem**, que envolverá a tentativa intencional de codificar uma transição proibida pela lógica do domínio da FSM. A documentação do erro de compilação resultante comprovará a **Hipótese de Segurança de Tipos**, validando que o sistema de tipos de Haskell é capaz de especificar transições de estado permitidas e proibidas (Perone, 2023) em tempo de compilação.

Por fim, a **Fase 4: Análise e Conclusão** consistirá na comparação qualitativa dos resultados do modelo funcional puro implementado com o modelo imperativo tradicional, focando na **Segurança de Tipos**, **Completude** e **Modularidade** alcançadas. O código-fonte desenvolvido servirá como a evidência principal para a elaboração final do relatório, que sistematizará o conhecimento adquirido e apresentará a validação conclusiva das hipóteses de pesquisa.

4 ESTADO DA ARTE

O desenvolvimento desta seção encontra-se em fase preliminar devido à necessidade de aprofundamento e identificação de referencial teórico específico que aborde a fusão dos domínios da Teoria de Autômatos, Programação Funcional Pura, Teoria das Categorias e o framework CREM.

A literatura revisada até o momento estabelece os fundamentos teóricos (FSMs, Monads, ADTs), mas a análise comparativa de soluções que aplicam formalismos de Categoria Kleisli para a composabilidade de Máquinas de Estados Finitos em arquiteturas de software ainda requer a exploração de artigos de conferência e periódicos de alta relevância, que podem ser menos acessíveis em bases de dados iniciais.

Dessa forma, a construção integral e conclusiva do Estado da Arte será realizada concomitantemente com o desenvolvimento do projeto. À medida que o artefato de software for implementado e novos referenciais técnicos forem analisados, esta seção será complementada com a devida contextualização da pesquisa em relação à fronteira do conhecimento atual.

5 PROGRESSO PARCIAL

Esta seção apresenta, de forma detalhada, os resultados alcançados no desenvolvimento do projeto, que seguiu uma abordagem estruturada para a implementação de Máquinas de Estados Finitos (FSMs) em Haskell. A exposição segue a ordem cronológica do trabalho, evidenciando

a correspondência explícita entre o código-fonte produzido e o arcabouço teórico que o fundamenta, e culmina na aplicação do framework CREM para a verificação formal das propriedades do sistema.

5.1 Correspondência entre Tipos de Dados Algébricos e os Conjuntos Q e Sigma

O primeiro resultado obtido consiste na tradução direta dos conjuntos finitos de Estados (Q) e Eventos (Σ) da quíntupla formal da FSM para Tipos Soma em Haskell, por meio do artefato `stateMachine.hs`:

```
data State = Locked | Unlocked deriving (Show, Eq)
data Input = Coin | Push deriving (Show, Eq)
```

Do ponto de vista categorial, essa codificação não é uma escolha arbitrária de sintaxe, mas a instanciação direta de um resultado estrutural bem estabelecido. Milewski (2020), ao apresentar a álgebra dos tipos, demonstra que o tipo `Bool`, definido em Haskell como `data Bool = True | False`, corresponde exatamente ao número 2, entendido como o coproduto de duas cópias do objeto terminal ($2 = 1 + 1$). Sob essa ótica, tanto `data State = Locked | Unlocked` quanto `data Input = Coin | Push` são, estruturalmente, isomorfos a `Bool`; ambos são coprodutos de dois objetos terminais na categoria Hask, a categoria cujos objetos são os tipos de Haskell e cujos morfismos são as funções entre eles (Milewski, 2020). Esse resultado confirma, em nível formal, que a finitude exigida pela definição matemática de Q e Σ é preservada com fidelidade pela representação em tipos, e não apenas sugerida por convenção de nomenclatura.

5.2 A Função de Transição delta como Morfismo em Hask

O segundo resultado é a implementação da função de transição $\delta : Q \times \Sigma \to Q$ por meio de *pattern matching* exaustivo:

```
transition :: State -> Input -> State
transition Locked   Coin  = Unlocked
transition Locked   Push  = Locked
transition Unlocked Coin  = Unlocked
transition Unlocked Push  = Locked
```

Observa-se que a assinatura `State -> Input -> State` é a versão *curried* da função. Milewski (2020) discute detalhadamente o processo de *currying*, segundo o qual uma função de múltiplos argumentos pode ser reescrita, sem perda de informação, como uma sequência de funções de um único argumento; em particular, uma função do tipo `(a, b) -> c` é equivalente, via *currying*, a `a -> b -> c`. Isso significa que `transition` é, de fato, a representação *curried* de um morfismo cujo domínio é o produto categorial `State` $\times$ `Input`, a

mesma noção de produto discutida por Milewski (2020) no contexto da construção universal de pares, o que estabelece uma correspondência direta e não apenas metafórica entre a assinatura de tipo do código e a definição formal $\delta : Q \times \Sigma \to Q$.

A verificação da Completude Funcional decorre diretamente do mecanismo de *pattern matching* de Haskell. Hutton (2016) descreve o *pattern matching* como um mecanismo em que "uma sequência de padrões sintáticos é usada para escolher entre uma sequência de resultados do mesmo tipo", sendo a definição da função dada por equações que cobrem casos distintos da estrutura do argumento. Mais importante para a hipótese de pesquisa é o princípio de segurança de tipos descrito por Hutton (2016) na definição do sistema de tipos de Haskell:

> "Porque a inferência de tipos precede a avaliação, os programas em Haskell são *type safe*, no sentido de que erros de tipo nunca podem ocorrer durante a avaliação. Na prática, a inferência de tipos detecta uma classe muito ampla de erros de programa, e é uma das características mais úteis de Haskell" (Hutton, 2016, tradução nossa).

Combinado com a exigência de exaustividade do *pattern matching*, a ausência de qualquer uma das quatro cláusulas de `transition` é sinalizada pelo compilador GHC como um caso não tratado, esse princípio garante que *todo* par válido $(\text{Estado}, \text{Evento})$ seja necessariamente coberto pela definição de δ antes mesmo de o programa ser executado. Esse resultado corrobora empiricamente a Hipótese de Completude Funcional, deslocando a verificação da cobertura lógica da FSM da fase de testes de *runtime* para o estágio de compilação.

5.3 Sequenciação Inicial e Validação Empírica

A composição de múltiplas transições foi inicialmente resolvida por meio da função de alta ordem `foldl`, aplicando repetidamente δ sobre uma lista de entradas:

```
run :: State -> [Input] -> State
run = foldl transition
```

A execução sobre a sequência `[Coin, Push, Push, Coin, Coin, Push]`, a partir do estado `Locked`, produziu deterministicamente o estado final `Locked`. Esse resultado, embora simples, é relevante porque a função `transition` utilizada aqui satisfaz a definição de função pura apresentada por Milewski (2020): "funções que sempre produzem o mesmo resultado dado o mesmo argumento de entrada, e não possuem efeitos colaterais". É justamente essa pureza, ausência de estado mutável de classe ou de variável global, na terminologia empregada por Malakhovski e Shalyto (2009), que viabiliza a aplicação direta da verificação de tipos e da exaustividade do *pattern matching* como mecanismos de corretude, algo estruturalmente inacessível ao modelo imperativo tradicional.

5.4 Da Composição Ingênua à Necessidade de uma Abstração Monádica

O resultado obtido na etapa anterior é suficiente apenas enquanto a função de transição não precisa carregar nenhuma informação adicional associada a cada mudança de estado (por exemplo, uma mensagem de saída ou um registro de auditoria da transição). No momento em que essa necessidade surge, como ficará evidente na aplicação do *framework* CREM, ao se examinar a assinatura das ações, a função de transição deixa de ser uma função pura simples do tipo $Q \times \Sigma \to Q$ e passa a ser o que Milewski (2020) denomina uma função "embelezada" (*embellished function*), do tipo $Q \times \Sigma \to M\,Q$, em que M é um *functor* monádico responsável por carregar o efeito computacional associado à transição (seja ele um registro, uma possibilidade de falha, ou qualquer outro contexto).

Esse foi o principal resultado teórico desta etapa: identificar, a partir do próprio artefato de código inicial, o ponto exato em que a composição via `foldl` deixaria de ser suficiente, e formalizar a solução por meio da Categoria Kleisli. Segundo Milewski (2020), uma categoria de Kleisli tem como objetos os tipos da linguagem subjacente, e seus morfismos entre um tipo A e um tipo B são funções que vão de A a um tipo derivado de B por meio de uma determinada "embelezagem"; cada categoria de Kleisli define sua própria forma de compor tais morfismos, bem como os morfismos de identidade correspondentes a essa composição. Formalmente, para um *functor* monádico M, os morfismos de Kleisli entre A e B têm o tipo $A \to M\,B$, com composição dada pelo operador infixo "*fish*":

```
(>=>) :: (a -> Writer b) -> (b -> Writer c) -> (a -> Writer c)
m1 >=> m2 = \x -> let (y, s1) = m1 x
                      (z, s2) = m2 y
                  in (z, s1 ++ s2)

return :: a -> Writer a
return x = (x, "")
```

Esse formalismo, apresentado por Milewski (2020) no contexto do *Writer monad*, é diretamente análogo ao problema enfrentado pelo projeto: assim como o *Writer* acumula uma trilha textual (*log*) a cada composição de funções, a FSM precisa acumular tanto o novo estado quanto uma eventual saída associada a cada transição, sem que essa responsabilidade seja delegada a uma variável mutável externa.

5.5 Reformulação Teórica da FSM sobre a State Monad

Para o tratamento específico do estado, e não apenas de um registro textual, o resultado teórico central foi mapear a FSM sobre a `State Monad` tal como formalizada por Hutton (2016). O autor define o transformador de estado como um tipo `newtype` que encapsula uma função pura de transformação de estado, evitando a necessidade de instâncias de classe sobre sinônimos de tipo:

```
newtype ST a = S (State -> (a,State))

app :: ST a -> State -> (a,State)
app (S st) x = st x

instance Functor ST where
  fmap g st = S (\s -> let (x,s') = app st s in (g x, s'))

instance Applicative ST where
  pure x = S (\s -> (x,s))
  stf <*> stx = S (\s -> let (f,s')  = app stf s
                             (x,s'') = app stx s'
                         in (f x, s''))

instance Monad ST where
  st >>= f = S (\s -> let (x,s') = app st s in app (f x) s')
```

A partir dessa estrutura, o projeto derivou a formalização de uma transição monádica para a FSM, em que o "estado" de Hutton (2016) é instanciado pelo estado do autômato (Q) e o valor produzido a cada passo é a saída associada à transição:

```
transicaoM :: Input -> ST Output
transicaoM evt = S (\q -> let q' = transition q evt
                           in (mensagem q evt, q'))
```

Essa reformulação segue exatamente o mesmo padrão estrutural do exemplo canônico de Hutton (2016) para geração de valores frescos durante o percurso de uma árvore (`fresh :: ST Int; fresh = S (\n -> (n, n+1))`), no qual um contador interno é encapsulado e roscado (*threaded*) de forma pura através de sucessivas aplicações, exatamente o comportamento requerido para que o estado do autômato seja propagado, transformado e extraído a cada evento da sequência, sem exposição de mutabilidade.

5.6 Sequenciação via Notação do e a Função mapM

Um segundo resultado foi a identificação, na literatura anexada, do mecanismo exato para generalizar a composição de `foldl` para o contexto monádico. Hutton (2016) define, de forma genérica para qualquer `Monad m`, a função:

```
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
mapM f [] = return []
mapM f (x:xs) = do y  <- f x
```

```
    ys <- mapM f xs
    return (y:ys)
```

Aplicando `mapM` à função `transicaoM` sobre a lista de eventos, obtém-se a ação monádica `mapM transicaoM eventos :: ST [Output]`, cuja execução por meio de `app (mapM transicaoM eventos) Locked` produziria simultaneamente (i) a lista de saídas associadas a cada transição e (ii) o estado final do autômato, generalizando, de maneira formalmente rigorosa, o resultado obtido apenas com `foldl`. A tradução dessa notação para o idioma do próprio projeto, "notação do", encontra respaldo direto em Oliveira (2017), que descreve essa construção como uma forma de reescrever expressões baseadas no operador de ligação (»=) "de uma maneira parecida com as linguagens de programação imperativas", preservando, porém, a pureza referencial subjacente.

5.7 Fundamentação Categorial: o Monad como Monoide na Categoria dos Endofuntores

Por fim, consolidou-se a fundamentação categorial mais abstrata do projeto. Milewski (2020) define formalmente um *monad* como um *endofunctor* T equipado com duas transformações naturais,

$$\mu : T \circ T \to T \qquad \eta : I \to T,$$

satisfazendo leis de associatividade e de unidade estruturalmente idênticas às leis de um monoide, resultado que o autor sintetiza na conhecida formulação atribuída a Saunders Mac Lane: "*monad is just a monoid in the category of endofunctors*" (Milewski, 2020). Esse resultado teórico é o que garante, em última instância, que a composição de transições por meio do operador *fish* (>=>) seja associativa e possua um elemento neutro (`return`), propriedade essencial para que a sequenciação de eventos da FSM produza o mesmo resultado independentemente de como a lista de eventos for particionada ou agrupada durante o processamento, generalizando com rigor formal a associatividade implicitamente exigida pelo uso de `foldl`.

Em síntese, o resultado desta etapa é de natureza primariamente teórica e formal: a FSM foi mapeada com precisão sobre o arcabouço da `State Monad` (Hutton, 2016) e da Categoria Kleisli (Milewski, 2020), estabelecendo a base necessária para a etapa seguinte. Notavelmente, essa fundamentação se revelou não apenas preparatória, mas estritamente necessária: como será demonstrado na aplicação do *framework* CREM, o próprio framework exige, na assinatura de suas ações, um contexto `Applicative m` genérico, evidência prática direta de que a generalização monádica discutida é pré-requisito conceitual para compreender e operar a arquitetura de composição estudada.

5.8 Estudo de Caso: Topologia Promovida ao Nível de Tipos

A etapa atual do projeto aplicou o *framework* CREM (*Composable Representable Executable Machines*), de Perone (2023), ao mesmo estudo de caso da catraca, com o objetivo de

investigar a Hipótese de Composabilidade e Modulação. O primeiro resultado foi a codificação da topologia de transições como um valor de primeira classe, posteriormente promovido ao nível de tipos por meio do mecanismo de *singletons*:

```
$(singletons [d|
  data TurnstileVertex = Locked | Unlocked
    deriving (Show, Eq)

  turnstileTopology :: Topology TurnstileVertex
  turnstileTopology = Topology
    [ (Locked,   [Unlocked])
    , (Unlocked, [Locked])
    ]
  |])
```

Essa construção representa uma extensão do sistema de tipos descrito por Hutton (2016), em que classes de tipos, como `Eq` e `Ord`, já restringem quais tipos podem ser usados em determinados contextos, levada a um nível mais expressivo: em vez de restringir apenas o *tipo* de um valor, a topologia restringe quais *valores* de transição são aceitáveis, tornando a regra de negócio da FSM uma propriedade do próprio sistema de tipos.

5.9 Verificação Formal de Segurança de Tipos em Tempo de Compilação

O resultado mais significativo é a verificação prática da Hipótese de Segurança de Tipos. As ações da FSM foram implementadas como:

```
turnstileAction :: Applicative m
  => STurnstileVertex initialVertex
  -> Input
  -> ActionResult m TurnstileTopology
     STurnstileVertex initialVertex Output
turnstileAction SLocked Coin =
  pureResult "Unlocked (insert coin)" SUnlocked
turnstileAction SLocked Push =
  pureResult "Locked (push denied)" SLocked
turnstileAction SUnlocked Coin =
  pureResult "Unlocked (already open)" SUnlocked
turnstileAction SUnlocked Push =
  pureResult "Locked (passed through)" SLocked
```

Note-se que a assinatura de `turnstileAction` confirma diretamente a conclusão teórica da etapa anterior: a ação é genérica sobre um contexto `Applicative m` arbitrário, e o

construtor `ActionResult` exige, implicitamente, uma prova em tempo de compilação (a restrição `AllowedTransition topology initialVertex finalVertex`) de que a transição declarada pertence à topologia especificada.

O princípio de *type safety* formulado por Hutton (2016), segundo o qual "erros de tipo nunca podem ocorrer durante a avaliação", pois a inferência de tipos precede a execução, foi aqui estendido do nível sintático para o nível semântico do domínio: uma tentativa deliberada de codificar uma transição fora da topologia declarada (por exemplo, `turnstileAction SLocked Push = pureResult "..."SUnlocked`) foi rejeitada pelo compilador GHC *antes* da execução do programa, por ausência da instância `AllowedTransition` correspondente. O erro de tipo, nesse caso, não decorre de uma incompatibilidade estrutural entre tipos primitivos (como discutido por Hutton, 2016, no contexto geral da linguagem), mas de uma violação de regra de negócio codificada como restrição de tipo, exatamente a proposta de Perone (2023) de especificar transições permitidas e proibidas diretamente na assinatura de tipos.

5.10 Execução e Validação Cruzada

A execução do artefato (via `cabal run`, dado que o CREM é uma dependência externa) sobre a mesma sequência de entradas `[Coin, Push, Push, Coin, Coin, Push]` produziu o seguinte traço:

```
Coin -> Unlocked (insert coin)
Push -> Locked (passed through)
Push -> Locked (push denied)
Coin -> Unlocked (insert coin)
Coin -> Unlocked (already open)
Push -> Locked (passed through)
```

O estado final resultante desse traço é `Locked`, idêntico ao estado final obtido de forma independente pela implementação ingênua (`run Locked inputs = Locked`). Essa coincidência não é trivial: trata-se de uma validação cruzada empírica entre duas formalizações estruturalmente distintas da mesma FSM, uma baseada em *pattern matching* simples com `foldl` e outra baseada em topologia promovida ao nível de tipos com efeito monádico genérico, reforçando a corretude semântica da função de transição δ modelada desde o início e corroborando a consistência do encadeamento teórico estabelecido.

5.11 Limitações Identificadas e Encaminhamentos

Em conformidade com o rigor científico exigido, registra-se uma limitação identificada na formalização atual: como o evento (`Input`) não está codificado no próprio vértice do tipo, a `Topology` declarada restringe apenas a alcançabilidade estado-a-estado ($Q \to Q$), e não o par completo $(Q \times \Sigma) \to Q$ exigido pela definição formal de δ. Uma direção metodológica

identificada para o refinamento dessa modelagem, a ser explorada na continuidade do trabalho, é o uso da noção de *functor* representável descrita por Milewski (2020), segundo a qual um *functor* pode ser interpretado como "um contêiner para armazenar resultados memoizados de chamadas de função", indexado por um tipo-chave. Essa técnica sugere um caminho formal para codificar o alfabeto Σ como índice (chave) das transições permitidas em cada vértice da topologia, aproximando ainda mais a representação em tipos da definição matemática completa de δ.

Adicionalmente, a geração automática de diagramas topológicos, prevista pelo módulo `Render` do CREM (Perone, 2023), ainda não foi exercitada, permanecendo como resultado pendente.

5.12 Síntese Geral e Validação Preliminar das Hipóteses

A síntese dos resultados obtidos permite uma sistematização da evolução do rigor formal alcançado em cada etapa do projeto. Inicialmente, na implementação ingênua, obteve-se um mecanismo de verificação de corretude baseado na exaustividade do *pattern matching*, com um mecanismo de composição simples via `foldl`, fundamentado teoricamente por Hutton (2016) no contexto de Tipos de Dados Algébricos. Em seguida, na etapa de formalização teórica, avançou-se para a utilização da `State Monad` e do operador *fish* (`>=>`), com base na Categoria Kleisli descrita por Milewski (2020), estabelecendo as leis do monoide/monad como fundamento para a verificação de corretude. Finalmente, na etapa de verificação formal com o framework CREM, alcançou-se o nível mais expressivo de segurança, com a prova `AllowedTransition` em tempo de compilação, conforme proposto por Perone (2023), generalizando o mecanismo de composição para os construtores `Sequential` e `Parallel`. Essa evolução evidencia que, enquanto nas etapas iniciais transições inválidas não eram detectáveis ou resultavam em indefinição, na etapa final tais violações são capturadas como erros de compilação.

Os resultados obtidos permitem uma avaliação preliminar das hipóteses de pesquisa. A Hipótese de Completude Funcional foi confirmada já na implementação inicial, por meio da exaustividade do *pattern matching* garantida pelo compilador (Hutton, 2016). A Hipótese de Segurança de Tipos foi validada em dois níveis: um nível básico, decorrente da própria natureza *type safe* de Haskell (Hutton, 2016), e um nível mais profundo, específico do domínio da FSM, comprovado empiricamente por meio da rejeição, em tempo de compilação, de transições não previstas na topologia (Perone, 2023). Por fim, a Hipótese de Composabilidade e Modulação teve sua fundamentação teórica estabelecida através da Categoria Kleisli e da `State Monad` (Hutton, 2016; Milewski, 2020), e sua primeira instância prática obtida por meio da arquitetura genérica de `ActionResult` do CREM; a demonstração completa dessa hipótese, contudo, depende ainda da composição efetiva de múltiplas FSMs independentes por meio dos construtores `Sequential`, `Parallel`, `Alternative` e `Feedback`, etapa que permanece como objetivo central para a continuidade do trabalho.

6 CRONOGRAMA

O cronograma a seguir descreve o planejamento de todas as etapas da pesquisa ao longo do período de desenvolvimento do projeto de Iniciação Científica.

Etapas	**fev**	**mar**	**abr**	**mai**	**jun**	**jul**	**ago**	**set**	**out**	**nov**	**dez**
Revisão bibliográfica aprofundada	X	X									
Modelagem de ADTs e função de transição		X	X								
Implementação de State Monad e Kleisli			X	X							
Estudo de caso e composição de FSMs				X	X						
Testes de invalidez de tipagem					X	X					
Análise comparativa e documentação						X	X				
Elaboração do relatório final							X	X	X		
Entrega e apresentação										X	X

7 REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS - ABNT. **NBR 6023**: Informação e documentação - Referências - Elaboração. Rio de Janeiro, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS - ABNT. **NBR 10520**: Informação e documentação - Citações em documentos - Apresentação. Rio de Janeiro, 2023.

BOCCHINO JR., R. L. **Monads, Categories, and Computation**. [S.l.]: [s.n.], 2025.

HUTTON, G. **Programming in Haskell**. 2. ed. Cambridge: Cambridge University Press, 2016.

JAHN, M. **Monads and their Applications in Haskell**. 2020. Trabalho de Conclusão de Curso (Bachelor's Thesis) – Department of Mathematics, Technische Universität München, Munich, 2020.

MAIA, D. **Applied Category Theory: Monads and Haskell**. 2022.

MALAKHOVSKI, Y. M.; SHALYTO, A. A. [Máquinas de estados finitos em linguagens de programação funcional pura]. RSDN Magazine, São Petersburgo, n. 1, 2009.

MILEWSKI, B. **Category Theory for Programmers**. [S.l.]: [s.n.], 2020. E-book.

OLIVEIRA, A. G. de. **Haskell: Uma introdução à programação funcional**. São Paulo: Casa do Código, 2017.

PERONE, M. Crème de la Crem: Composable Representable Executable Machines. 2023.

WADLER, P. Comprehending Monads. *Mathematical Structures in Computer Science*, v. 2, n. 4, p. 461-493, 1992.